Auto-Scaling for Cloud Microservices

Ashley Davis, Albert Dinh, Roberto Villarreal Andrade

Table of Contents

Table of Contents **1**
1. Abstract **2**
2. Introduction **2**
3. Technologies, Methodologies & Tools **3**
3.1. Terraform 3
3.2. Docker 3
3.3. Python 3
3.3.1. Docker SDK 3
3.3.2. Locust 3
4. Design Artifacts **4**
4.1. Algorithm 4
4.1.1. Response Time 4
4.1.2. Average Response Time 4
4.1.3. Scaling Calculations 5
4.1.4. Monitor 5
4.2. User Stories 6
4.2.1. Scale Up 6
4.2.2. Scale Down 6
4.3. Class Diagram 6
4.4. High-Level Architectural Diagram 7
4.5 State Diagram 7
4.6 Flow Diagram 8
4.7 Pseudocode 8
4.8 Real Time Plot 9
5. Deployment Instructions & User guide **9**
5.1. Infrastructure Setup 9
5.1.1. VPN 9
5.1.2. Option One: Project Started Kit 9
5.1.3. Option Two: Terraform 9
5.1.4. Load Balancer 10
5.1.5. Testing (Bell Curve) 10
6. Conclusion **11**
7. References **11**

1. Abstract

This report outlines the design and development of a reactive auto-scaling engine for a cloud microservice application deployed on the Cybera infrastructure using Docker microservices. Using Python and Docker SDK, the auto-scaler engine continually monitors response times of a web microservice. Based on the average response time, the engine will dynamically horizontally-scale the docker microservice based on predefined lower and upper response time thresholds. Overall, the goal of the engine is to optimize performance by minimizing the response time for users as well as the operational cost on the cloud.

2. Introduction

Scaling in cloud computing is essential for efficient infrastructure management. Scaling refers to the ability to increase or decrease resources as needed to meet changing demand [1]. This eliminates the need to over-provision resources upfront to handle peak levels of user activity [2], thereby minimizing operational costs. Horizontal scaling refers to the ability for an application to automatically scale by adding or reducing computing nodes according to the workload.

This report will explore the design and development of a reactive auto-scaling engine for a cloud microservice application deployed on the Cybera infrastructure using Docker microservices. The auto-scaler engine continually monitors response times of a web microservice. Based on the average of these response times, the engine will dynamically horizontally-scale the docker microservice based on predefined lower and upper response thresholds. If the average response time exceeds the upper threshold, the engine will scale up the web application to optimize performance. If the average response time succeeds the lower threshold, the engine will scale down the web application to optimize operational costs.

3. Technologies, Methodologies & Tools

3.1. Terraform

Terraform was used to provision and manage the cloud infrastructure hosted on Cybera. By using the OpenStack provider, we were able to provision security groups rules, keypairs and instances.

3.2. Docker

Docker swarm mode was used in both the swarm manager VM and swarm node VM. This allowed declaring the number of replicas for the web microservice, scaling the service up or down. The swarm manager automatically distributes the replicas between both VMs.

3.3. Python

Python was used as the main programming language for the auto-scaling engine. The specific Python version of our autoscaler is Python 3.8.

3.3.1. Docker SDK

The main reason for choosing Python was due to its Docker Engine API support. The Docker SDK for Python library was used to interact with the Docker engine to scale the web service.

3.3.2. Locust

Locust was chosen as the main load testing tool. Locust allowed testing the auto-scaler engine with a bell-shaped workload. Additionally it provided vital information such as average response time and charts for total request per second, response times and number of users.

4. Design Artifacts

4.1. Algorithm

4.1.1. Response Time

The auto-scaling engine retrieves the response time of the Docker web service by doing an HTTP GET request and timing the response as showing in the following figure:

```python
t0 = time.time()
requests.get(f"http://{self.config['host']}",
timeout=(self.config["upper_threshold"] * 2))
t1 = time.time()
```

The request timeouts if the response takes longer than double the predefined upper threshold. If this occurs, the response time is noted down as 2*upper_threshhold for future calculations. This method was chosen to avoid unresponsive and hanging responses.

4.1.2. Average Response Time

As the engine retrieved response times, the average response time was calculated by dividing the sum of all response times by the total of all response times as shown in *ResponseTimeList.average()* method where ResponseTimeList is shown below:

```python
class ResponseTimeList:
  def __init__(self, max_size):
    self.max_size = max_size
    self.list = []

  # will delete oldest entry to maintain max_size length
  def append(self, time):
    self.list.append(time)
    if len(self.list) > self.max_size:
      self.list.pop(0)

  def average(self):
    return sum(self.list) / len(self.list)
```

Figure 2. ResponseTimeList source code

The maximum capacity of the array that stores the time responses is six (6) as shown in the variable *time_array_length* in the *config.yml* shown in the following code snippet:

```
Unset
time_array_length: 6
```

For this report's reference, config.yml is shown as following:

```
Unset
poll_interval: 5 # 10 5 2 3 10
time_array_length: 6
min_replicas: 1
max_replicas: 30
service_name: app_name_web
host: "10.2.9.11:8000"
monitor_periods: 2
# Acceptable response time
upper_threshold: 4.5
lower_threshold: 2
```

4.1.3. Scaling Calculations

The number of replicas to scale is determined dynamically by an equation. For scaling up, the number of replicas to add equals the average response time divided by the upper response threshold, rounded to the nearest integer. For scaling down, the number of replicas to remove equals the lower threshold divided by the average response time, rounded to the nearest integer. This allows the number of replicas to increase or decrease by multiple replicas in proportion to how much the average response time exceeds or subceeds the allowed thresholds. This equation was determined through testing and experimentation.

4.1.4. Monitor

In order to avoid oscillation, the engine was set to monitor the response times for multiple polling intervals after scaling. This allows the engine to find a better response time average for future scale operations and prevents it from scaling after every polling interval.

4.2. User Stories

4.2.1. Scale Up

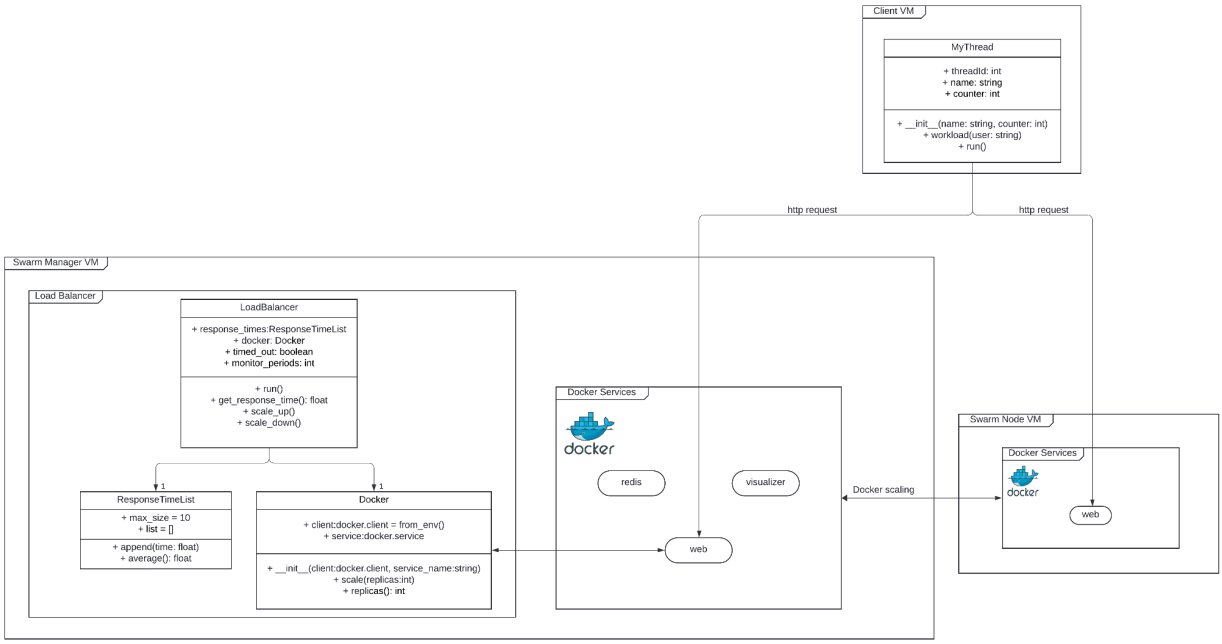
"As a user, I want to continuously be able to get a response from the system within an acceptable amount of time, so that I can access the resource quickly despite how many other users are requesting the same service."

4.2.2. Scale Down

"As a system owner, I want to maintain the minimum number of replicas running at a time while still providing users with acceptable response times so that operation costs are reduced but users are able to get a response from the system in an acceptable amount of time."

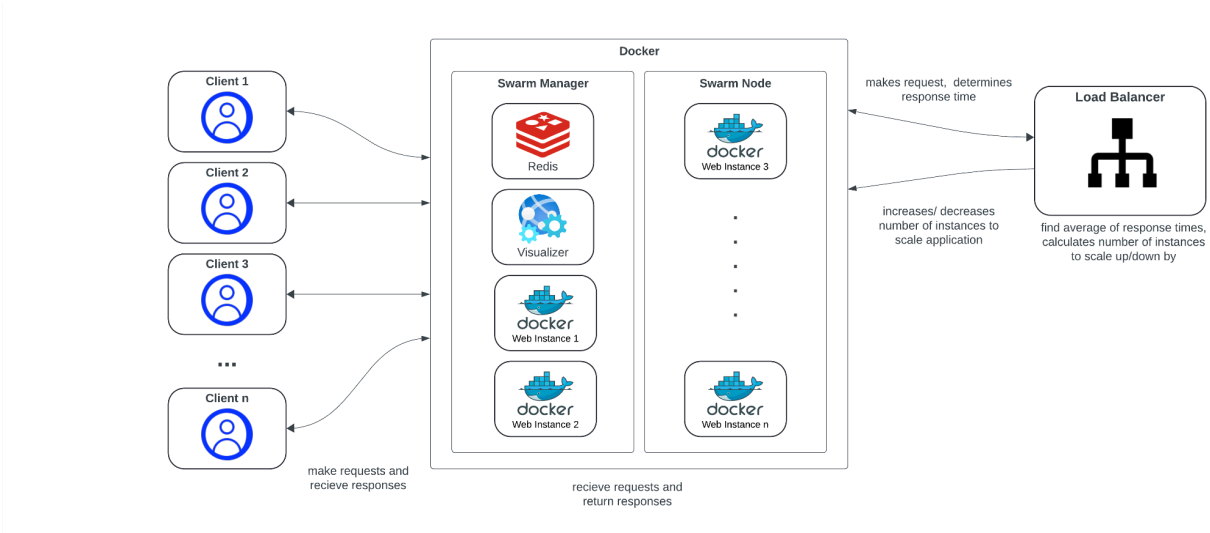
4.3. Class Diagram

The following class diagram shows the structures, relationships, and interactions of the different classes of the application.



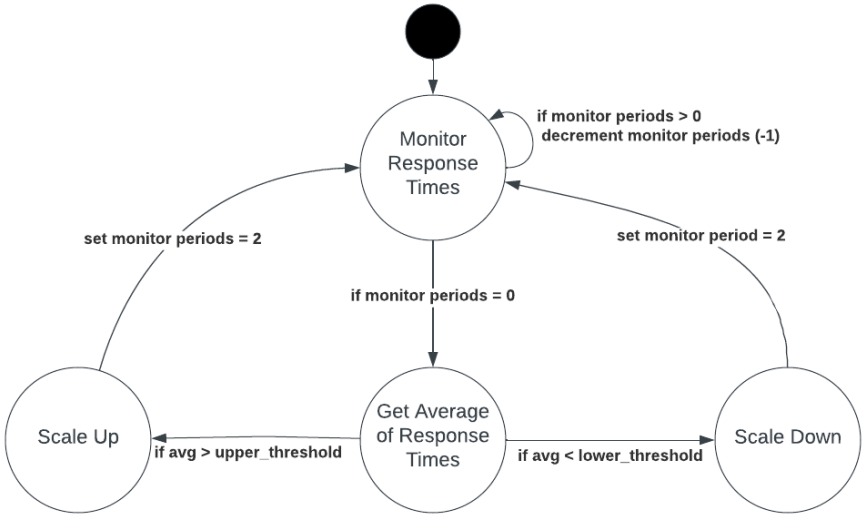
4.4. High-Level Architectural Diagram

This diagram shows the high-level interactions between the application, clients, and load balancer. There can be any number of clients (0 - n), and any number of Docker web instances (in range from min_replicas to max_replicas as defined in the load balancer configuration file config.yml).



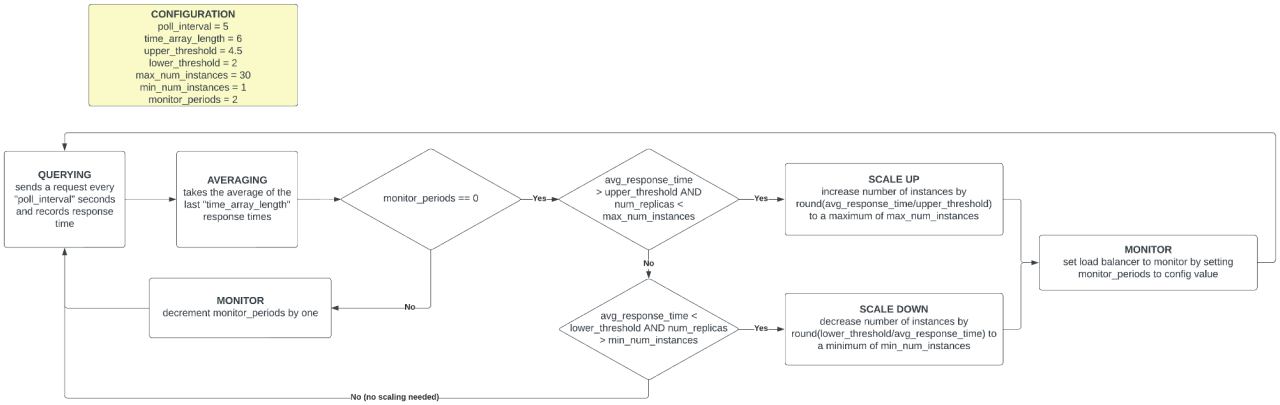
4.5 State Diagram

The following diagram shows the different states of the load balancer and the progression between them due to variables and calculations.



4.6 Flow Diagram

The following diagram shows the logic of the load balancer as it progresses through states, as well as the calculations that are performed. It also shows an example of the configuration values.



4.7 Pseudocode

The following pseudocode outlines the process of the load balancer script.

```
Unset
Start of Program

Set the configuration variables
    poll_interval
    min_replicas
    max_replicas
    service_name
    host_IP
    upper_threshold
    lower_threshold
    num_reqs_for_average
    monitor_intervals

Set reponse_times_array[num_reqs_for_average]

define get_response_time()
    make request to host
    return time_after_request - time_before_request

loop:
    response_times_array.append(get_reponse_time())
    avg_response_time = average(reponse_times_array)

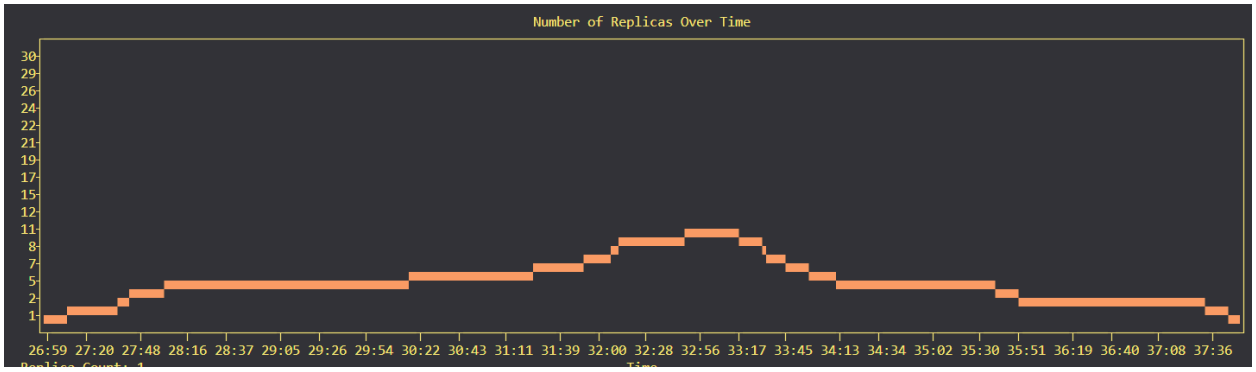
    if no need to wait intervals:
        if avg_response_time > upper_threshold and num_active_dockers < max_replicas:
            scale_up(num_reps_to_add = round(avg_response_time / upper_threshold))
            then monitor for 2 intervals
        else if avg_response_time < lower_threshold and num_active_dockers > min_replicas:
```

```
            scale_down(num_reps_to_remove = round(lower_threshold / avg_response_time))
            then monitor for 2 intervals
    else
        monitor_intervals - 1

    sleep(poll_interval);
    plot number of replicas
End of Program
```

4.8 Real Time Plot

The load balancer displays a real time plot of the number of replicas over time in the terminal.



5. Deployment Instructions & User guide

5.1. Infrastructure Setup

5.1.1. VPN

Follow the instructions to set up the Cybera VPN. The VPN is **required** to SSH into the virtual machines as well as to view the web pages.

5.1.2. Option One: Project Started Kit

Follow the project starter kit instructions.

5.1.3. Option Two: Terraform

1. Navigate to the Cybera dashboard and select Compute → API Access → Download OpenStack RC file v3.
2. Clone the GitHub repository and navigate to the cybera/terraform directory.
3. Run the OpenStack RC file (.sh extension)

4. Run “terraform init” then “terraform apply”
5. SSH into the Swarm Manager VM and run the following commands:
    a. sudo docker swarm init (will return token)
    b. sudo docker stack deploy --compose-file docker-compose.yml app_name
6. SSH into the Swarm Node VM and run the following command:
    a. docker swarm join --token xxxxxxxxxxxxxxxxx swarm_manager_ip:2377

5.1.4. Load Balancer

1. Git clone the repository into the VM

```
Unset
git clone
git@github.com:rob-villarre/ECE-422-Auto-Scaling-for-Cloud-Microservices.git
```

2. Navigate to the directory

```
Unset
cd ECE-422-Auto-Scaling-for-Cloud-Microservices/load-balancer
```

3. Activate the environment and run the python script

```
Unset
source ./venv/bin/activate/
```

4. Run the load balancer script

```
Unset
sudo python3 main.py
```

5. Turn off auto-scaling using the following command argument

```
Unset
sudo python3 main.py --scale off
```

5.1.5. Testing (Bell Curve)

6. Navigate to the directory

```
Unset
cd ECE-422-Auto-Scaling-for-Cloud-Microservices/testing
```

7. Activate the environment

```
Unset
source ./venv/bin/activate/
```

8. Run the locust script - localhost:8089

```
Unset
locust
```

6. Conclusion

This project demonstrated the use of an auto-scaling/load-balancing program that increases and decreases the number of Docker images used to handle requests. This allowed the Docker image and its replicas to maintain response times within designated thresholds, which ensured that users would be provided with adequate response times but that the system also did not over-provision the number of replicas (therefore decreasing operational costs). The tests for the auto-scaler showed that the response times of the system were maintained within the thresholds while the number of users or requests per second fluctuated.

7. References

[1] https://www.vmware.com/ca/topics/glossary/content/cloud-scalability.html
[2] https://aws.amazon.com/what-is-cloud-computing/
[3] https://locust.io/

Diagrams created with LucidChart - https://www.lucidchart.com/